

OASIS-X

SWIFT FHS — Smart Waveform Integrated Fibre-optic Transmission Fault Healing System

System Architecture & UML Diagrams

Prepared as Supplementary Material for Project Defence

Project: Final Year Project — Computer Engineering
System: Autonomous Optical Fibre Network Self-Healing
Repository: github.com/jaycode100-sys/OASIS-X
Date: June 09, 2026

Contents

- Fig 1 — System Architecture (5-Layer)
- Fig 2 — UML Class Diagram
- Fig 3 — Sequence Diagram: Run Pipeline
- Fig 4 — Use Case Diagram
- Fig 5 — Module Dependency Graph

This document is a technical supplement to the submitted project report.

FIG 1 Figure 1 — System Architecture

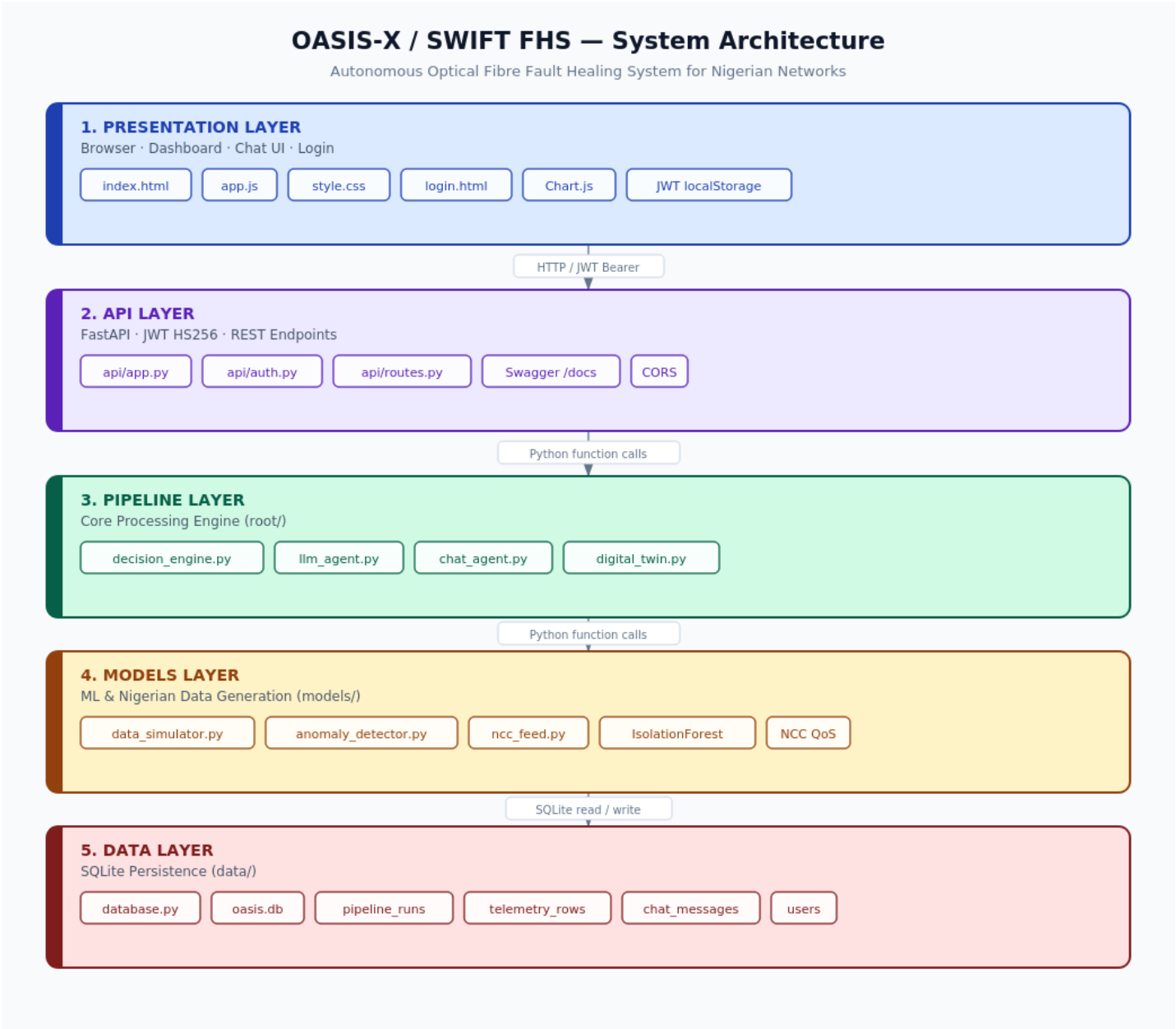


Figure 1 — System Architecture

Five-layer architecture showing the Presentation Layer (browser dashboard), API Layer (FastAPI + JWT), Pipeline Layer (decision engine, LLM agent, digital twin), Models Layer (IsolationForest anomaly detection, NCC compliance), and Data Layer (SQLite persistence). Layers communicate via HTTP/JWT at the top and Python function calls internally.

FIG 2 Figure 2 — UML Class Diagram

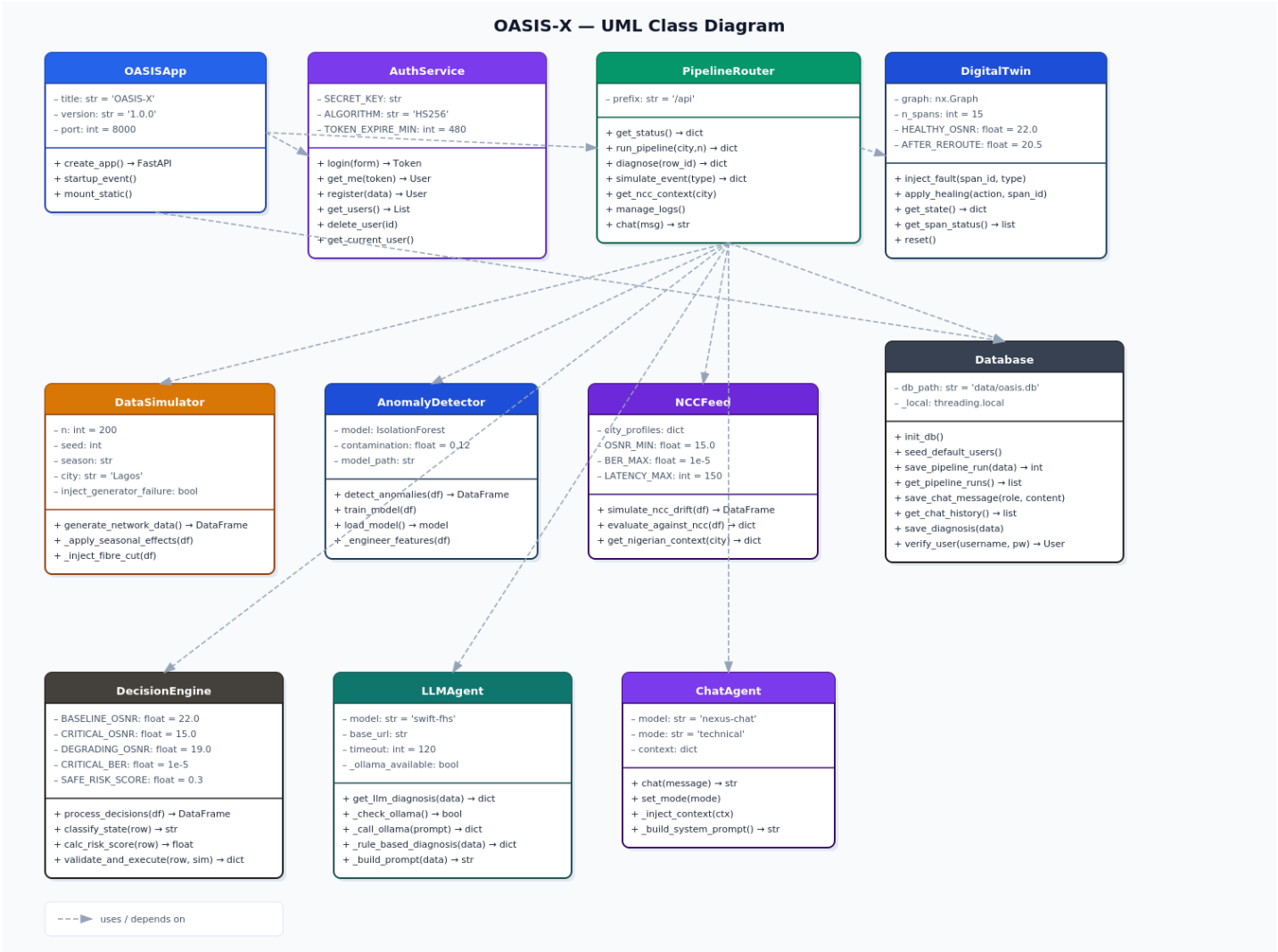


Figure 2 — UML Class Diagram

Key classes across all modules: OASISApp (entry point), AuthService (JWT auth), PipelineRouter (all endpoints), DataSimulator (Nigerian telemetry generation), AnomalyDetector (IsolationForest, contamination=0.12), NCCFeed (NCC QoS compliance), DecisionEngine (state classification + action planning), LLMAgent (swift-fhs + rule fallback), ChatAgent (Nexus)

FIG 3 **Figure 3 — Sequence Diagram: Run Pipeline**

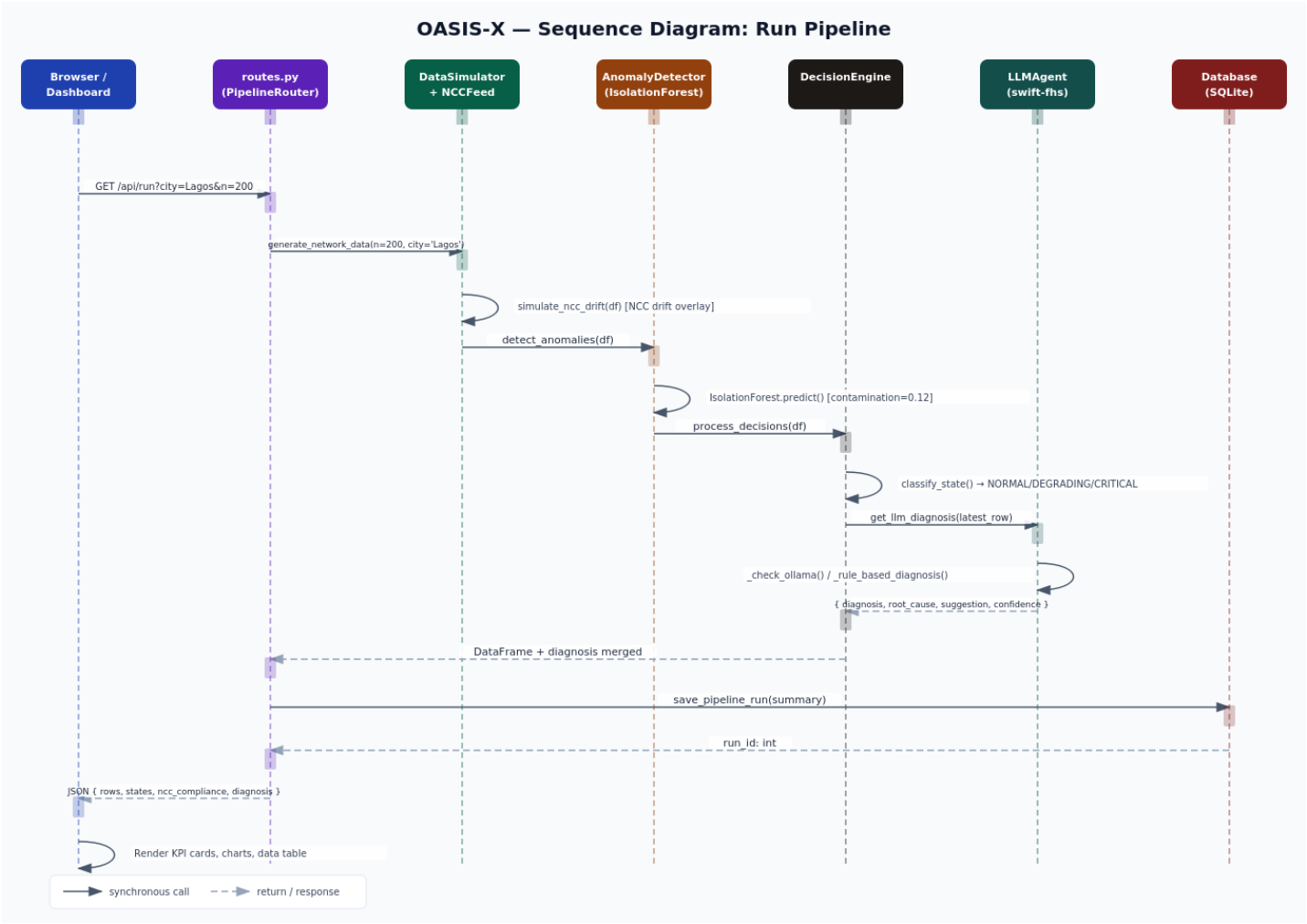


Figure 3 — Sequence Diagram: Run Pipeline
Interaction flow for a single pipeline execution: the browser sends GET /api/run, which triggers data generation, NCC drift simulation, IsolationForest anomaly detection, decision engine state classification, optional LLM diagnosis (with rule-based fallback), database persistence, and a JSON response rendered on the dashboard. Solid arrows = synchronous calls; dashed

FIG 4 **Figure 4 — Use Case Diagram**

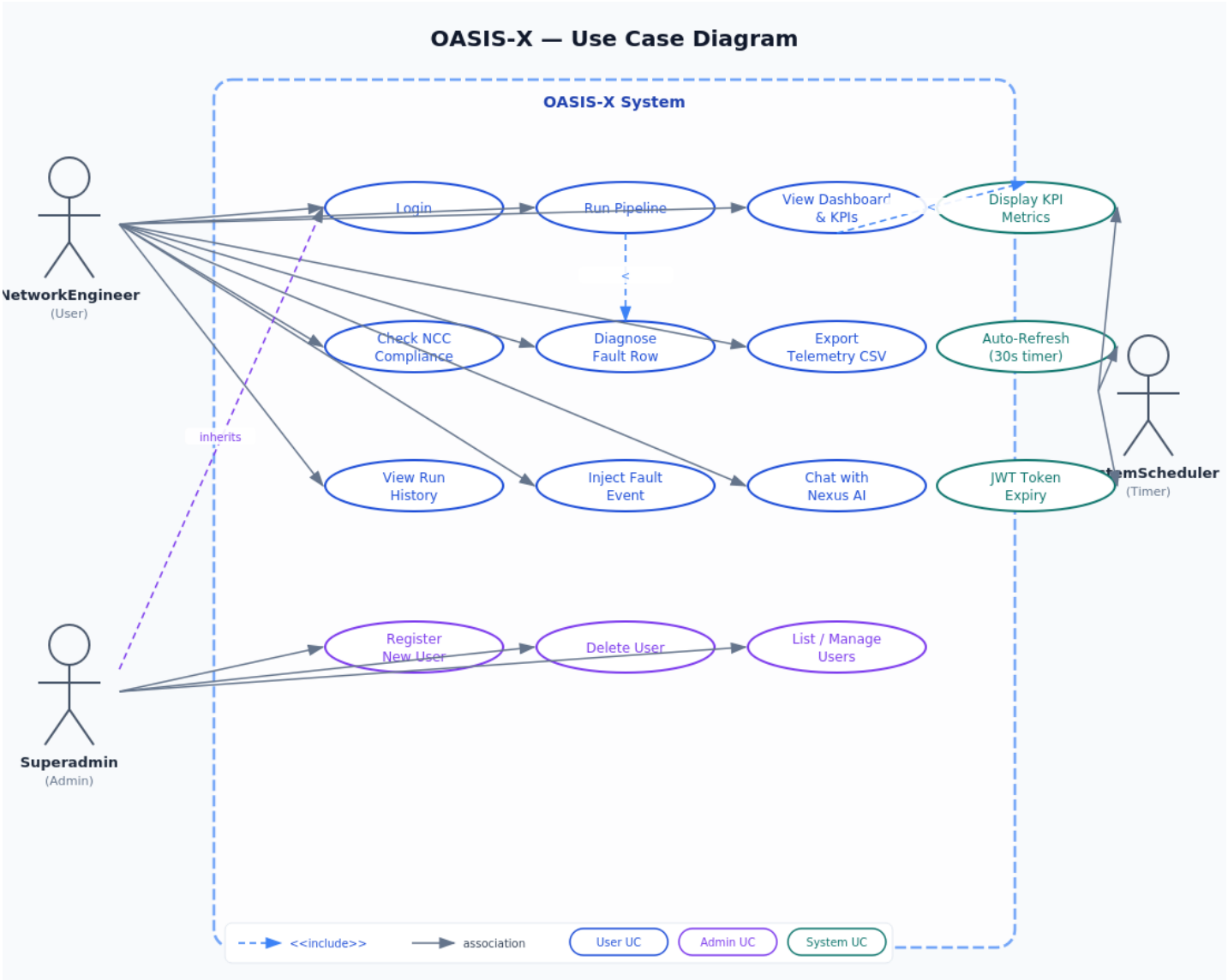


Figure 4 — Use Case Diagram

Three actors: Network Engineer (User), Superadmin, and System Scheduler. User-accessible use cases include login, run pipeline, view NCC compliance, diagnose fault rows, inject fault events, chat with Nexus AI, export CSV, and view run history. Superadmin extends User with user management capabilities (register, list, delete). System use cases include KPI auto-refresh

FIG 5 **Figure 5 — Module Dependency Diagram**

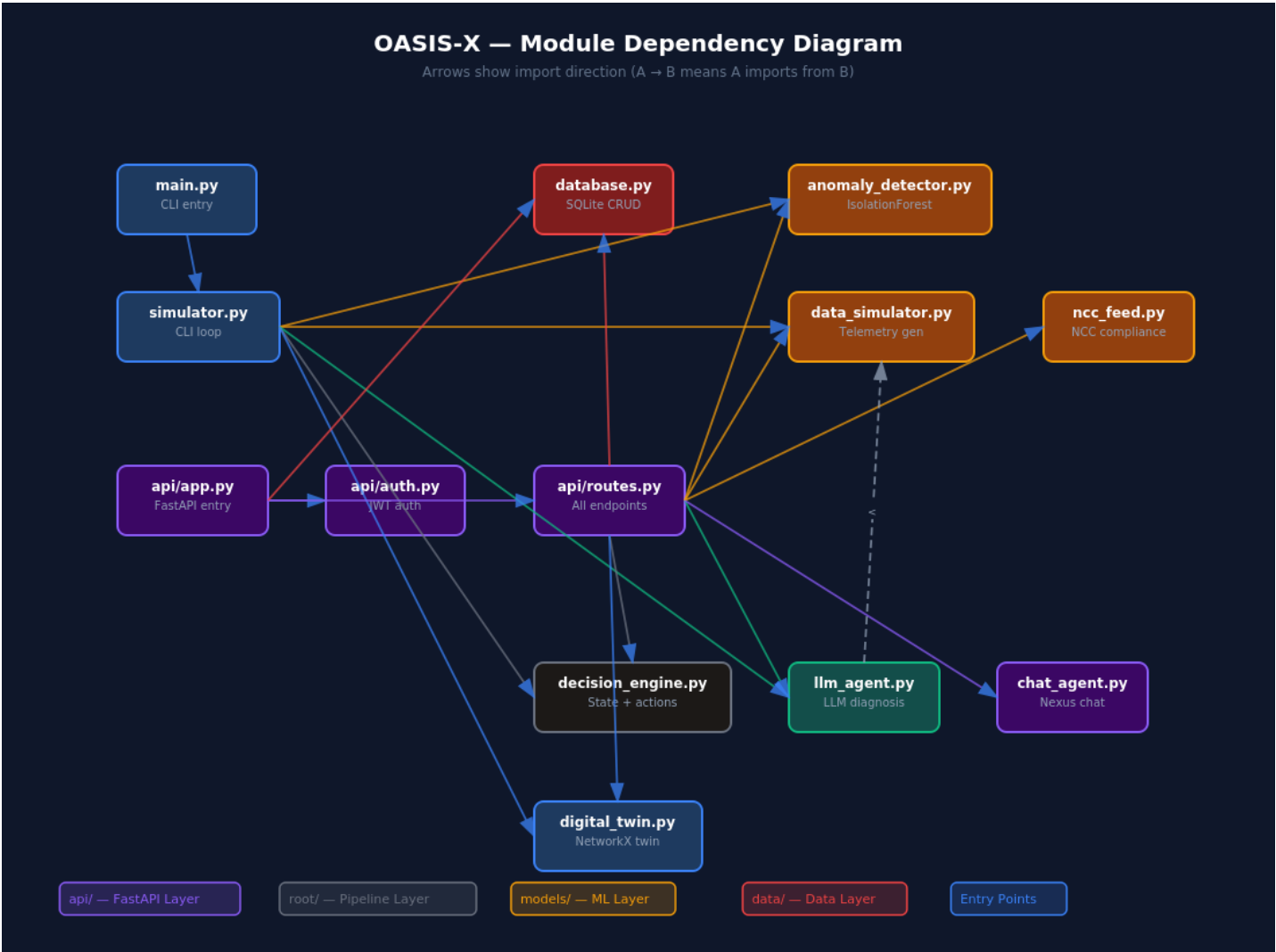


Figure 5 — Module Dependency Diagram

Import graph showing how modules depend on each other. `api/routes.py` is the central hub, consuming all pipeline layer modules (`decision_engine`, `llm_agent`, `chat_agent`, `digital_twin`) and all model layer modules (`data_simulator`, `anomaly_detector`, `ncc_feed`) as well as the database. The CLI entry point (`main.py` → `simulator.py`) uses the same pipeline and